Problem statement:

You have been given two datasets. One dataset includes application data for every customer that has been given a loan in a 6 month period The other contains every loan that has been given in this time and whether it has been a good loan or a bad loan. Use the data to identify which new applicants should be given a loan in the future. Discuss any problems or limitations with the data, shortcomings of the analysis and/or additional data elements that might be useful. You should present your findings in a presentation lasting less than 60 minutes including time for questions.

Objective:

1. Clean and preprocess the data.
2. EDA
3. Merge the application and loan performance dataset.
4. Build a model to predict for upcoming users.

Step 1: Import Required Libraries

I started by importing the necessary Python libraries for data analysis, visualization, preprocessing, and machine learning.

- Pandas and NumPy: Data manipulation and numerical calculations.
- Matplotlib and Seaborn: Data visualization and exploratory data analysis.
- Scikit-learn: Data preprocessing, model building, and evaluation.
- Joblib: Model persistence and saving trained models.

Step 2: Load the Datasets

I loaded two Excel datasets:

- Application_data.xlsx — contains customer and loan application information.
- Loan_performance.xlsx — contains loan performance records and the loan outcome indicator.

I examined the first few records, dataset dimensions, column data types, and overall structure to understand the available features and identify potential data quality issues.

Step 3: Merge the Datasets

I extracted the customer identifier from the idLoan column in the loan performance dataset by splitting the loan ID and standardizing the extracted identifier to lowercase.

I then merged the application and loan performance datasets using customer_id with an inner join.

This step combined customer application details with historical loan outcomes into a single dataset for analysis.

Step 4: Create the Target Variable

I converted the existing flgGood loan status into a numerical target variable named is_good.

- 1 represents a Good Loan.
- 0 represents a Bad Loan.

This binary target variable is used to train supervised classification models. I also checked the proportion of good and bad loans to understand the distribution of the target classes.

Step 5: Perform Data Quality Checks

I assessed the merged dataset for missing values, data types, numerical distributions, and duplicate loan IDs.

These checks helped me understand the quality and completeness of the data before feature engineering and model development.

Step 6: Perform Exploratory Data Analysis (EDA)

I performed exploratory data analysis to understand the relationship between customer characteristics and loan performance.

The analysis included:

- Comparing the number of good and bad loans.
- Evaluating the ROC-AUC of individual raw credit score features.
- Comparing good-loan rates across categorical variables, such as residence type, payment frequency, bank account details, and loan duration.

- Comparing numerical variables across good and bad loans using median values and box plots.
- Analyzing income, age, and credit score quartiles.
- Examining correlations between numerical features and the target variable.
- Checking changes in good-loan rates over application months.

This step helped identify potentially useful predictors and understand which features might distinguish good loans from bad loans.

Step 7: Perform Feature Engineering

I created additional features to represent customers' financial situations more effectively.

- Age: Calculated from the application date and birth date.
- Loan-to-income ratio: Approved loan amount divided by monthly income.
- Payment-to-income ratio: Payment amount divided by monthly income.
- Rent-to-income ratio: Monthly rent divided by monthly income.

For ratio calculations, zero income was treated as missing to avoid division-by-zero errors.

These engineered features provide additional information about a customer's financial commitments relative to their income.

Step 8: Handle Missing Values

I handled missing values using different approaches depending on the feature.

- Filled missing payment_amount_approved values using payment_amount.
- Replaced missing values in how_use_money and bank_account_duration with "Unknown".
- Replaced missing other_phone_type values with "None".

During model training, I used median imputation for numerical features and most-frequent-value imputation for categorical features. This allowed the preprocessing pipeline to handle remaining missing values systematically.

Step 9: Prepare the Features and Target

I separated the independent variables from the target variable.

- Features (X): Customer, financial, application, loan, and credit score attributes.

- Target (y): The binary is_good variable.

I excluded identifiers, the original target label, and fields such as email, bank routing number, ZIP code, and application dates from the model inputs.

I also retained the original categorical columns in the modeling dataset so that they could be processed appropriately by the machine learning pipeline.

Step 10: Split the Dataset into Training and Testing Sets

I divided the data into training and testing sets using an 80:20 split.

Because the dataset contains application dates, I used a chronological split when the dates were available and valid. Earlier applications were used for training, while later applications were reserved for testing.

This approach helps evaluate how well the model performs on applications from a later period rather than relying only on a random split.

Step 11: Build the Preprocessing Pipeline

I created a reusable preprocessing pipeline to ensure that data preparation was applied consistently.

- Numerical features were imputed using the median.
- Categorical features were imputed using the most frequent value.
- Categorical features were transformed using one-hot encoding, with unseen categories handled safely.
- Numerical features were standardized for Logistic Regression.

I combined preprocessing and model training in a single pipeline to reduce the risk of inconsistent transformations between training and testing data.

Step 12: Train and Compare Machine Learning Models

I trained and compared three classification algorithms:

1. Logistic Regression

A linear classification model used as a baseline to evaluate relationships between the input features and loan outcomes.

2. Random Forest Classifier

An ensemble of decision trees that can capture nonlinear patterns and interactions between features.

3. Extra Trees Classifier

An ensemble learning algorithm that uses randomized decision trees to capture complex relationships in the data.

I used class weighting to help address potential class imbalance and compared the models using several evaluation metrics.

Step 13: Evaluate Model Performance

I evaluated the trained models using the following metrics:

Metric	Purpose
ROC-AUC	Measures the model's ability to distinguish between good and bad loans across thresholds.
Average Precision	Summarizes precision-recall performance, particularly useful when classes are imbalanced.
Accuracy	Measures the proportion of correct predictions.
Precision	Measures how many predicted good loans are actually good.
Recall	Measures how many actual good loans are correctly identified.
F1-score	Balances precision and recall.

I compared the models using ROC-AUC and Average Precision, along with the other classification metrics, to identify a promising model for further optimization.

Step 17: Save the Trained Model

Finally, I saved the complete optimized preprocessing and classification pipeline using Joblib as:

taplend_loan_pipeline.joblib

Saving the entire pipeline preserves the preprocessing steps and trained model together, making it easier to apply consistent transformations and generate predictions on new data.

The notebook also includes code to generate predictions and predicted probabilities for new observations.

4. Conclusion

The project follows an end-to-end machine learning workflow, beginning with data integration and exploratory analysis, followed by feature engineering, preprocessing, model comparison, evaluation, and model saving.